

StreetBoardman — Engineer / App Manager Manual

This is the technical reference for developers and future app managers. For the product design rationale (why decisions were made the way they were), see [ARCHITECTURE.md](#) — this manual focuses on "how do I actually work with this codebase."

1. Architecture

```
React SPA (Vite, mobile-first) --REST/JSON, cookies--> Express API
|
| Prisma ORM
|
| PostgreSQL
|
| (production only) Paystack webhook
```

- **Frontend:** React 18 + React Router, no UI framework — plain CSS designed mobile-first (see `client/src/styles.css`). Three route trees (`/better` , `/boardman` , `/admin`), each gated by `ProtectedRoute` .
- **Backend:** Node.js + Express, layered as `routes -> middleware -> controllers -> services -> Prisma` .
- **Database:** PostgreSQL via Prisma ORM (`prisma/schema.prisma`).
- **Auth:** JWT access + refresh tokens, stored in httpOnly cookies (not localStorage, to reduce XSS exposure).
- **Payments:** Paystack, integrated but gated behind `APP_MODE=PRODUCTION` — see section 8.
- **Wallet system:** an append-only ledger (`WalletTransaction`); a wallet's `balance` is always derived from applying transactions, never edited directly (`server/services/walletService.js`).
- **Betting engine:** `server/services/bettingService.js` — validates, deducts stake, creates the bet, all inside one DB transaction.
- **Result engine:** `server/services/resultService.js` — the state machine for `PENDING_CONFIRMATION -> CONFIRMED/DISPUTED` , plus the scheduled auto-confirm sweep (`server/jobs/autoConfirmSweep.js`).
- **Payout engine:** `server/services/payoutService.js` — pari-mutuel split of the pool, commission calculation, idempotent by design.

2. Project Structure

```
streetboardman/
|-- client/                React app
```

```

| | | -- src/
| | | | -- routes/
| | | | | -- auth/ Landing, Login, Register*
| | | | | -- better/ Every Better-facing screen
| | | | | -- boardman/ Every Boardman-facing screen
| | | | | -- admin/ Every Admin-facing screen
| | | | -- components/ ProtectedRoute, BottomNav, TopBar
| | | | -- api/client.js fetch wrapper (credentials: 'include')
| | | | -- context/AuthContext.jsx current user/session state
| | | | -- App.jsx all route definitions
| | | | -- styles.css the entire mobile-first design system
| | | -- public/ manifest.json, sw.js (PWA)
| |
| -- server/
| | -- app.js Express app: middleware + route mounting
| | -- index.js process entry point, starts the cron job
| | -- routes/ one file per resource, thin route wiring
| | -- controllers/ request/response shaping only
| | -- services/ ALL business logic and money math lives here
| | -- middleware/ auth, requireRole, validate, ratelimit, errorHandler, auditLog
| | -- validators/schemas.js every Zod request-body schema
| | -- utils/ password, jwt, idGenerator, money, appError, asyncHandler
| | -- jobs/autoConfirmSweep.js node-cron sweep, runs every minute
| | -- config/ env.js, db.js (Prisma client), constants.js
| |
| -- prisma/
| | -- schema.prisma the full data model
| | -- seed.js demo accounts + a sample competition
| |
| -- tests/
| | -- unit/ pure-function tests (no DB)
| | -- integration/ supertest + real Postgres
| | -- helpers/reset.js wipes the DB between tests
| |
| -- docs/ this file + the other manuals
|-- package.json backend deps + scripts

```

Rule enforced throughout the codebase: controllers never touch Prisma directly for anything financial, and never contain money math. That always lives in `server/services/*Service.js`. If you're adding a feature and find yourself writing `+ / -` on a wallet balance inside a controller, stop — it belongs in `walletService.applyWalletTransaction`.

3. Database

See `prisma/schema.prisma` for the authoritative source. Summary of every model and why it exists:

Model	Purpose	Key fields
User	Every human account, discriminated by <code>role</code>	<code>role</code> , <code>phone</code> (unique), <code>passwordHash</code> , <code>status</code>

Model	Purpose	Key fields
BoardmanProfile	Boardman-only extension of User	approvalStatus , commissionRateOverride
Wallet	One per user, plus one singleton PLATFORM wallet	walletType , balance
WalletTransaction	Append-only ledger; the ONLY source of truth for balance changes	type , amount (signed), balanceBefore , balanceAfter
Competition	One local event with betting attached	status , boardmanCommissionRate / platformCommissionRate
CompetitionParticipant	The sides/players in a competition	name
BetOption	A choice Betters can pick (e.g. "Tunde wins")	totalStaked (running total)
Bet	One Better's stake on one option	betCode (unique, e.g. SB-000123), stake , status
Result	The Boardman's submitted outcome + confirmation state	status , confirmationDeadline , winningSide
Payout	One payment to one winning bet	idempotencyKey (unique) — prevents double-paying
Commission	The commission split calculated for one competition	boardmanAmount , platformAmount
Deposit	Money coming in (demo or Paystack)	provider , providerReference (unique)
Withdrawal	Money requested out	status , destination (JSON bank details)
Dispute	A flagged result, freezing payouts	status , resolvedByAdminId

Method	Path	Auth	Notes
GET	/users/me	any	Returns user + boardmanProfile if applicable
PATCH	/users/me	any	Update fullName/email
GET	/wallet/me	any	Current balance
GET	/wallet/me/transaction	any	Full ledger for this user

Deposits / Withdrawals

Method	Path	Auth	Notes
POST	/deposits/demo	BETTER/BOARDMAN	Instant demo credit
POST	/deposits/paystack/in	BETTER	403 unless APP_MODE=PRODUCTION
POST	/deposits/paystack/webhook	Paystack only (signed)	Raw body, HMAC-verified, idempotent
GET	/deposits/me	any	Deposit history
POST	/withdrawals	BETTER/BOARDMAN	Debits wallet immediately, PENDING
GET	/withdrawals/me	any	Withdrawal history

Competitions / Bets

Method	Path	Auth	Notes
GET	/competitions	none	Public list of BETTING_OPEN

Method	Path	Auth	Notes
GET	/competitions/:id	none	Public detail
POST	/competitions	BOARDMAN, approved	Creates with BETTING_OPEN status
GET	/competitions/mine/li	BOARDMAN	Own competitions, any status
PATCH	/competitions/:id/close	BOARDMAN, owner	BETTING_OPEN -> BETTING_CLOSED
GET	/competitions/:id/bet	BOARDMAN, owner	Bets received
POST	/competitions/:id/result	BOARDMAN, owner	BETTING_CLOSED -> PENDING_CONFIRMATION
POST	/competitions/:id/dis	any authenticated	PENDING_CONFIRMATION -> DISPUTED
POST	/bets	BETTER	{ betOptionId, stake }, rate-limited
GET	/bets/me	BETTER	Own bet history
GET	/bets/:betCode	BETTER, owner	One ticket

Admin (all require role: ADMIN)

Method	Path	Notes
GET	/admin/overview	Platform-wide counts + revenue
GET	/admin/boardmen / /admin/boardmen/pending	List
PATCH	/admin/boardmen/:id/approve /reject /suspend	
GET	/admin/users	

Method	Path	Notes
PATCH	<code>/admin/users/:id/suspend</code> <code>/reactivate</code>	
GET	<code>/admin/competitions</code> / <code>/admin/bets</code>	Full visibility
GET	<code>/admin/disputes</code>	
PATCH	<code>/admin/disputes/:id/resolve</code>	<code>{ action: 'CONFIRM' 'CANCEL', winningOptionId? }</code> — runs the payout or refund engine
PATCH	<code>/admin/withdrawals/:id/process</code> <code>/reject</code>	
GET	<code>/admin/ledger</code>	Recent <code>WalletTransactions</code> across all wallets
GET	<code>/admin/audit-logs</code>	
GET	<code>/admin/settings</code> / PATCH <code>/admin/settings</code>	Commission rates, confirmation window

Error format: every error response is `{ "error": "human-readable message" }` with an appropriate HTTP status (400/401/403/404/409/422/500). Validation errors (422) come from the Zod schemas in `server/validators/schemas.js`.

5. Financial Architecture

```

Deposit (demo instant, or Paystack webhook-verified)
  -> WalletTransaction(DEPOSIT) [Better/Boardman wallet += amount]
Bet placed
  -> WalletTransaction(BET_STAKE) [Better wallet -= stake] (same DB tx as Bet creation)
Result auto-confirmed or Admin-confirmed
  -> payoutService.processPayoutsForCompetition:
      commissionService.calculateCommission(pool, boardmanRate, platformRate)
      commissionService.calculateWinnerPayouts(winningBets, winningOptionTotal, distributablePool)
      for each winner: WalletTransaction(BET_WIN)
      WalletTransaction(COMMISSION) -> Boardman wallet
      WalletTransaction(COMMISSION) -> Platform wallet
Withdrawal requested
  -> WalletTransaction(WITHDRAWAL) [wallet -= amount immediately, held as PENDING]
Withdrawal rejected

```

```
-> WalletTransaction(ADJUSTMENT) [funds returned, with a reason note]
```

Core invariant: `wallet.balance` is only ever changed inside

`walletService.applyWalletTransaction`, via a single atomic `UPDATE ... WHERE balance + delta >= 0` — this is what prevents a race between two concurrent debits from ever pushing a balance negative, and it's also the only place a `WalletTransaction` row gets created. Every other file that needs to move money calls this function; none of them touch `wallet.balance` directly.

Idempotency points: - `Payout.idempotencyKey ( payout:<betId> )` — a retried payout transaction can't create a duplicate payout row. - `processPayoutsForCompetition` first does an atomic `updateMany({ where: { status: 'RESULT_CONFIRMED' }, data: { status: 'PAYOUT_PROCESSING' } })` — if two triggers race (e.g. the cron sweep and a manual admin action), only one gets `count: 1` and proceeds; the other sees `count: 0` and exits early. - `Deposit.providerReference` is unique — `handlePaystackChargeSuccess` checks `status === 'SUCCESSFUL'` before crediting, so a retried webhook is a no-op.

6. Result / Verdict Engine (implementation detail)

See ARCHITECTURE.md section 10 for the design rationale. Implementation lives in

`server/services/resultService.js`: - `submitResult` — Boardman-only, requires `BETTING_CLOSED`, sets `confirmationDeadline = now + resultConfirmationWindowHours`. - `raiseDispute` — any authenticated user, only while `PENDING_CONFIRMATION`. - `autoConfirmDueResults` — called every minute by `server/jobs/autoConfirmSweep.js`; finds every result past its deadline still `PENDING_CONFIRMATION` and confirms + pays out. - `resolveDispute` — Admin-only, `CONFIRM` (optionally overriding the winning option) or `CANCEL` (refund via `payoutService.cancelAndRefundCompetition`).

7. Environment Variables

See `.env.example` for the full annotated list. Highlights: - `APP_MODE` — `DEMO` or `PRODUCTION`. Real-money Paystack routes 403 in `DEMO`. **Never flip this to PRODUCTION without completing the legal and payment review** referenced in the original product brief. - `DATABASE_URL` — Postgres connection string. - `JWT_ACCESS_SECRET` / `JWT_REFRESH_SECRET` — must be long, random values in production, never the placeholders shipped in `.env.example`. - `PAYSTACK_SECRET_KEY` / `PAYSTACK_WEBHOOK_SECRET` — only needed once `APP_MODE=PRODUCTION`. - `DEFAULT_BOARDMAN_COMMISSION_RATE`, `DEFAULT_PLATFORM_COMMISSION_RATE`, `DEFAULT_RESULT_CONFIRMATION_WINDOW_HOURS` — seeded into `SystemSetting` on first run; editable afterwards from Admin -> Settings.

8. Deployment

1. **Database:** provision a PostgreSQL instance. Set `DATABASE_URL`.
2. **Migrate:** `npx prisma migrate deploy` (production) or `npx prisma migrate dev` (local development, creates migration files).

3. **Seed** (optional, demo accounts only): `npm run seed`.
4. **Backend**: `npm install && npm start` (or run under a process manager like PM2 / systemd). Set all variables from `.env.example` in the real environment — never commit `.env`.
5. **Frontend**: `cd client && npm install && npm run build`, then serve `client/dist` from a static host or CDN. Point it at the API's public URL (update `CLIENT_ORIGIN` on the backend to match the frontend's origin, for CORS).
6. **HTTPS**: required in production — cookies are marked `secure` when `NODE_ENV=production`, so they will not be sent over plain HTTP.
7. **Paystack webhook**: register `https://<your-domain>/api/deposits/paystack/webhook` in the Paystack dashboard, and set `PAYSTACK_WEBHOOK_SECRET` to match. Only switch `APP_MODE=PRODUCTION` once this is verified working.
8. **Backups**: schedule regular `pg_dump` backups of the Postgres database — this holds every wallet balance and transaction; treat it with the care of a real financial ledger.
9. **Logging/monitoring**: `morgan` logs every HTTP request; pipe `console.error` output (from `errorHandler.js` and the cron sweep) to your platform's log aggregator. Add uptime monitoring on `/health`.
10. **Domain**: point your domain at the frontend host; configure the backend behind a reverse proxy (nginx, or your host's equivalent) that terminates TLS and forwards to the Node process.

9. Maintenance

- **Updating the app**: pull changes, `npm install` in both `streetboardman/` and `streetboardman/client/`, run any new Prisma migrations (`npx prisma migrate deploy`), restart the Node process, rebuild and redeploy the client.
- **Running migrations**: `npx prisma migrate dev --name <description>` locally to create a new migration from schema changes; commit the generated `prisma/migrations/*` folder.
- **Inspecting logs**: `morgan` request logs + `console.error` output from the error handler and the cron sweep are your first stop for any incident.
- **Troubleshooting failed payments**: check the `Deposit` row for the reference in question — `status` tells you if Paystack ever confirmed it. Cross-check against Paystack's own dashboard for that transaction. Never manually flip a `Deposit.status` to `SUCCESSFUL` without also crediting the wallet through `depositService.handlePaystackChargeSuccess` — doing it directly in the DB breaks the ledger's invariant.
- **Handling failed withdrawals**: use `withdrawalService.rejectWithdrawal` (not a direct DB edit) — it returns the held funds via a proper `ADJUSTMENT` transaction with an audit trail.
- **Resolving disputes**: Admin -> Disputes screen, or directly via `resultService.resolveDispute` if scripting an emergency fix.
- **Recovering from system errors**: because every money movement is a `WalletTransaction` row, you can always reconstruct "what should this balance be" by summing transactions for a wallet and comparing to its stored `balance` — they should always match exactly.

- **Backup & restore:** standard `pg_dump` / `pg_restore` against the `DATABASE_URL` database; test restores periodically, not just backups.

10. Security Checklist (production)

- [] `JWT_ACCESS_SECRET` / `JWT_REFRESH_SECRET` are long, random, and unique per environment (never reused from `.env.example`).
- [] `NODE_ENV=production` so cookies are marked `secure`.
- [] HTTPS is enforced end-to-end (frontend, API, webhook).
- [] `CLIENT_ORIGIN` is set to the exact production frontend origin (CORS).
- [] `PAYSTACK_WEBHOOK_SECRET` matches what's configured in the Paystack dashboard, and the webhook route is verified to reject bad signatures (see `tests/`).
- [] `APP_MODE` is only set to `PRODUCTION` after legal/regulatory and payment provider review is complete.
- [] Database backups are scheduled and restore-tested.
- [] Rate limiting (`authLimiter` , `bettingLimiter`) is active — check `server/middleware/rateLimit.js` thresholds are appropriate for expected traffic.
- [] No `.env` file is committed (`.gitignore` already excludes it — verify before every deploy).
- [] Admin accounts are created directly in the database/seed script only — confirm no public registration path ever reaches `role: ADMIN`.
- [] Audit logs (`AuditLog`) are being written for every admin action — spot check after any change to `adminService.js`.